



---

v Publications using TU Darmstadt's Corporate Design (TUDaCI)-0 <00

---

# A Learned Advisor for Automated Materialized View Selection

---

Master thesis by Michael Girges

Date of submission: November 14, 2025

1. Review: Prof. Dr. Carsten Binnig
  2. Review: Johannes Wehrstein
  3. Review: Roman Heinrich  
Darmstadt
- 



TECHNISCHE  
UNIVERSITÄT  
DARMSTADT



Computer Science  
Department  
Data Management Lab

---

---

## **Erklärung zur Abschlussarbeit gemäß §22 Abs. 7 und §23 Abs. 7 APB der TU Darmstadt**

---

Hiermit versichere ich, Michael Girges, die vorliegende Masterarbeit ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt zu haben. Alle Stellen, die Quellen entnommen wurden, sind als solche kenntlich gemacht worden. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Fall eines Plagiats (§38 Abs. 2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

Bei der abgegebenen Thesis stimmen die schriftliche und die zur Archivierung eingereichte elektronische Fassung gemäß §23 Abs. 7 APB überein.

Bei einer Thesis des Fachbereichs Architektur entspricht die eingereichte elektronische Fassung dem vorgestellten Modell und den vorgelegten Plänen.

Darmstadt, 14. November 2025

---

Signature

---

# Abstract

---

Materialized View (MV) are commonly used in databases to accelerate read queries, but their benefits come with high maintenance costs and full refreshes to avoid stale data. Incremental Materialized View (IMV) offer an alternative by updating views incrementally through triggers. However, they perform best in read-heavy workloads and may introduce significant write-time overhead. This work proposes a machine-learning-based advisor that, given a SQL workload, predicts whether to use MV, IMV, or no views at all for optimal performance. The advisor jointly considers the cost of maintaining views during writes and the potential performance gains during reads. Our approach aims to guide database administrators and automated tuning systems in selecting the most effective view strategy based on workload characteristics.

---

# Contents

---

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>List of Figures</b>                                        | <b>5</b>  |
| <b>List of Abbreviations</b>                                  | <b>6</b>  |
| <b>1. Introduction</b>                                        | <b>8</b>  |
| 1.1. Context and Motivation . . . . .                         | 8         |
| 1.2. Problem Statement . . . . .                              | 8         |
| 1.3. Goals and Contributions . . . . .                        | 9         |
| 1.4. Thesis Outline . . . . .                                 | 9         |
| <b>2. Background</b>                                          | <b>10</b> |
| 2.1. Automated View Selection . . . . .                       | 10        |
| 2.2. Query Performance Prediction Using Cost Models . . . . . | 11        |
| 2.2.1. Tuple Time Tree (T3) Cost Model . . . . .              | 12        |
| 2.3. Workload Generation . . . . .                            | 12        |
| <b>3. Design</b>                                              | <b>13</b> |
| 3.1. Objectives . . . . .                                     | 13        |
| 3.2. Proposed Approach . . . . .                              | 13        |
| 3.2.1. System Architecture . . . . .                          | 14        |
| 3.2.2. Feature Engineering . . . . .                          | 14        |
| 3.2.3. Cost Model Development and Training . . . . .          | 15        |
| 3.2.4. Advisor Integration and Evaluation . . . . .           | 15        |
| <b>4. Implementation</b>                                      | <b>17</b> |
| 4.1. Implementation Overview . . . . .                        | 17        |
| <b>5. Evaluation</b>                                          | <b>18</b> |
| 5.1. Experimental setup . . . . .                             | 18        |
| <b>6. Conclusion and Future Work</b>                          | <b>19</b> |
| 6.1. Conclusion . . . . .                                     | 19        |
| 6.2. Future Work . . . . .                                    | 19        |
| <b>Bibliography</b>                                           | <b>20</b> |
| <b>Appendices</b>                                             | <b>22</b> |
| A. Experiment Parameters . . . . .                            | 22        |



---

## List of Figures

---

---

## List of Abbreviations

---

|               |                                                 |
|---------------|-------------------------------------------------|
| <b>API</b>    | Application Programming Interface               |
| <b>CQ</b>     | Completion Queue                                |
| <b>DBMS</b>   | Database Management System                      |
| <b>DMA</b>    | Direct Memory Access                            |
| <b>HPC</b>    | High Performance Computing                      |
| <b>IB</b>     | InfiniBand                                      |
| <b>MPI</b>    | Message Passing Interface                       |
| <b>NIC</b>    | Network Interface Controller                    |
| <b>OLAP</b>   | On-line Analytical Processing                   |
| <b>OLTP</b>   | On-line Transactional Processing                |
| <b>QP</b>     | Queue Pair                                      |
| <b>RAM</b>    | Random Access Memory                            |
| <b>RDMA</b>   | Remote Direct Memory Access                     |
| <b>RNIC</b>   | RDMA-enabled Network Interface Controller       |
| <b>RoCE</b>   | RDMA over Converged Ethernet                    |
| <b>RQ</b>     | Receive Queue                                   |
| <b>RR</b>     | Receive Request                                 |
| <b>SDN</b>    | Software Defined Networking                     |
| <b>SQ</b>     | Send Queue                                      |
| <b>SR</b>     | Send Request                                    |
| <b>TLB</b>    | Translation Lookaside Buffer                    |
| <b>TCP/IP</b> | Transmission Control Protocol/Internet Protocol |
| <b>MV</b>     | Materialized View                               |
| <b>IMV</b>    | Incremental Materialized View                   |
| <b>ML</b>     | Machine Learning                                |

---

**ILP** Integer Linear Programming

**DTA** Database Tuning Advisor

**T3** Tuple Time Tree



---

# 1. Introduction

---

---

## 1.1. Context and Motivation

---

MVs are a common optimization in database systems: by precomputing and storing query results they can substantially accelerate read queries. However, materialized views introduce maintenance costs whenever base tables change, because the view contents must be kept consistent with the underlying data. The naive strategy of full recomputation can become prohibitively expensive for workloads with frequent updates.

Incrementally maintained materialized views (IMVs) provide an alternative maintenance strategy by applying only the changes (deltas) caused by updates to base tables, instead of recomputing entire views. IMVs can significantly reduce maintenance work for many workloads, but they are not a universal win: their benefit depends on workload characteristics. In particular, IMVs tend to be most advantageous for read-heavy workloads with relatively small or structured updates, while write-heavy workloads can cause high incremental maintenance overhead and complex update propagation.

Choosing between no views, conventional MVs, and IMVs therefore requires trading off read-performance gains against the cost of maintaining views under updates. Manually exploring this trade-off is time-consuming and error-prone. This thesis proposes to automate that decision by building a machine learning-based advisor that, given a SQL workload composed of reads and writes, recommends the view strategy that is likely to be optimal in terms of overall system performance and maintenance cost.

---

## 1.2. Problem Statement

---

The central problem addressed in this work is:

Given a SQL workload (a sequence or distribution of read and write statements) and information about the database/schema, determine whether the workload is best served by (i) no materialized views, (ii) materialized views with full recomputation, or (iii) incrementally maintained materialized views (IMVs), taking into account both query performance and view maintenance costs.

Concretely, this requires (a) modelling the costs and benefits of each strategy for a given workload, and (b) designing a predictor that maps workload characteristics to the recommended strategy. To support cost-aware decision making, this thesis includes the design of a write-cost estimation model inspired by decision-tree approaches (cf. the T3 system[12]).

---

This thesis answers the following research questions:

1. When is it beneficial to use MV, IMV, or no materialized view at all for a given SQL workload?
2. Can a machine learning model accurately and efficiently predict the optimal view strategy for a workload based on its read and write characteristics?

---

### 1.3. Goals and Contributions

---

The goals of this thesis are to design, implement, and evaluate an automated advisor that recommends one of the three view strategies (no view, MV, or IMV) for a given SQL workload. The primary contributions are:

- The design of a machine learning-based advisor that takes workload features as input and outputs a recommended view strategy (no view / MV / IMV).
- A cost estimation component for write operations that informs the advisor; this component leverages a decision-tree-based approach inspired by the T3 system[12].
- An empirical evaluation demonstrating the advisor’s effectiveness at selecting strategies that balance read performance and maintenance overhead across a variety of workloads.

---

### 1.4. Thesis Outline

---

The remainder of the thesis is organized as follows:

- Chapter 2 presents background material and related work on materialized views, incremental maintenance techniques, and workload-aware tuning approaches.
- Chapter 3 describes the design of the ML advisor, including the workload feature representation and the cost estimation model for writes.
- Chapter 4 details the implementation decisions and integration with the chosen database/testing infrastructure.
- Chapter 5 reports the experimental methodology and results evaluating prediction accuracy and overall performance impact of the recommended strategies.
- Chapter 6 concludes the thesis, summarizes findings, and outlines directions for future work.

---

## 2. Background

---

---

### 2.1. Automated View Selection

---

Automated database tuning, notably the selection and management of physical design structures such as MVs and indexes, has long been a critical and challenging task for database administrators. Traditional approaches often rely on manual expertise [20] or on complex rule-based and enumeration-based heuristics, which become particularly cumbersome for rich MV structures or complex queries [1].

Early automated MV selection work explored heuristic systems that choose candidate views based on estimated cost and validate them against the actual database [1]. Commercial systems like Microsoft SQL Server's Database Tuning Advisor (DTA) provide workload-driven recommendations for indexes and MVs [1]. Other research formulates view selection as an optimization problem (for example, Integer Linear Programming (ILP)) [20]; while these formulations can be precise, they are often computationally expensive for large sets of subqueries or extensive workloads, and iterative methods may not guarantee convergence.

More recently, Machine Learning (ML) methods have been applied to a variety of database tasks with the goal of replacing handcrafted components by learned models that can automatically handle complexity and variability [8, 5]. A common paradigm in learned database components is workload-driven learning, which typically requires executing representative workloads on the target database to collect training data. Although effective in many settings, this data-collection step is costly and must often be repeated for each new database or substantially changed workload [8]. To address this limitation, zero-shot learning approaches have emerged: models are pretrained once and designed to generalize to unseen databases or workloads without per-database retraining. Initial zero-shot work has focused on tasks such as cost estimation [15], and researchers are investigating extensions of this paradigm to tasks like MV selection.

The maintenance of MVs - especially incremental maintenance - is another important axis in the literature. For example, commercial systems include refresh/maintenance modules that track refresh methods and schedules; some research and product directions explore using ML to predict refresh behavior or maintenance cost and to incorporate those predictions into recommendation logic [1]. This line of work suggests that predicted maintenance cost (including incremental refresh cost) can and should influence which views are recommended. The interaction between specific maintenance strategies (e.g., incremental refresh) and ML-based recommendation processes remains an active area where further research can provide valuable insights.

---

## 2.2. Query Performance Prediction Using Cost Models

---

Accurate query performance prediction is essential for effective physical design tuning, including MV selection. Traditional cost models, such as those used in query optimizers, estimate query execution time based on factors like I/O costs, CPU costs, and network costs [13]. However, these models often struggle to capture the complexities of modern database systems and workloads, leading to inaccuracies in performance predictions. The first cost model was proposed by Listgarten et al. [10] as part of their work on main memory databases. Moreover, a framework for building cost models for hierarchical database systems was introduced by Manegold et al. [11].

In the last couple of decades, learning-based techniques have been increasingly applied to the problem of query performance prediction. Gupta et al. pioneered the use of machine learning for query performance prediction by introducing a decision tree-based approach to predict query execution time ranges [3]. Query performance prediction has also been approached for static and dynamic workloads using regression models [2], who implemented these techniques on top of PostgreSQL. Calibration of existing cost models in PostgreSQL has been explored by Wu et al. [16], where they used small calibration workloads to adjust the cost model parameters. In combination with an analytical model, they were able to predict the query execution time for larger workloads [17]. This approach proved competitive to other contemporary learning-based methods.

Neural networks and deep learning techniques have also been employed for query performance prediction through feeding the tree-structured query plans into neural network architectures. For instance, Sun et al. [15] concluded that combining tree nodes is effectively handled using a pooling architecture. Zhou et al. proposed encoding single and multiple concurrent queries using a graph embedding approach [22]. This approach captures the interactions between concurrent queries and enables accurate predictions of execution times of whole workloads. Additionally, transformer-based architectures and augmenting the input data with database statistics have been shown to further improve prediction accuracy [21].

Since workload-based models do not work well in zero-shot settings, recent research has focused on developing zero-shot query performance prediction models. The first zero-shot query performance prediction model was proposed by Hilprecht et al. [4]. Their zero-shot model was trained and evaluated on different databases and workloads, demonstrating the ability to generalize to unseen databases without retraining. More recently, Wu et al. [18] introduced a zero-shot model that is trained on the complete Redshift workload. They propose a hierarchical model in an effort to improve latency.

Learning-based approaches have shown promising results in query performance prediction, but they are often criticized in literature for their high end-to-end latency. This latency can be a significant drawback in scenarios where quick predictions are essential for decision-making. Lehmann et al. [9] commented on this issue by concluding that learning-based query optimizers have high optimization times, which renders them impractical for real-world usage. One approach to mitigate this issue is proposed by Wu et al. [18], where they use a query cache with a fallback to a local decision tree model to reduce the end-to-end latency of their zero-shot model.

Other non-learning-based approaches have been proposed to solve the query performance prediction problem. Since these approaches are not dependent on neural networks, they can be more interpretable and can be quicker to train and evaluate. For instance, cost models based on differential programming have been proposed by Hilprecht et al. [6]. They depend on learning some of the constants in

---

pre-existing cost model functions. Another approach is to use different groups of calibration constants for the fixed cost functions in the PostgreSQL cost model [19].

### 2.2.1. Tuple Time Tree (T3) Cost Model

Rieger et al. proposed the Tuple Time Tree (T3) cost model [12], which bridges the gap between high-accuracy, high-latency learning-based models and their lower-accuracy, low-latency counterparts. T3 is a hybrid cost model that combines a low-latency decision tree with pipeline-based query plan presentation and tuple-centric prediction targets.

The pipeline-based query plan representation divides the query plan into multiple pipelines, where each pipeline consists of a sequence of operators that can be executed without materializing intermediate results. This representation allows T3 to generate a flat feature vector for each pipeline and predicts its execution time individually. The total query execution time is calculated as the sum of all pipeline predictions.

On the other hand, tuple-centric prediction is based on the idea that, instead of predicting the total execution time for a pipeline, T3 predicts the expected time it takes to process a single tuple within that pipeline. Then, the per-tuple time is multiplied by the pipeline's input cardinality to estimate the total execution time of the pipeline. This approach is highly compatible with the decision tree architecture and is highly-scalable.

Furthermore, T3 improves inference time by compiling the trained model directly into native machine code using the `l1eaves` framework. This step drastically reduces the average prediction latency from 22 $\mu$ s in interpreted trees and 1ms in zero-shot neural networks down to 4 microseconds.

---

## 2.3. Workload Generation

---

In this work, we require a diverse set of training and evaluation workloads to train and assess our proposed models. Workload generation is a well-researched area, with various methods proposed to create synthetic workloads that mimic real-world query patterns. For example, Kipf et al. [7] introduced a method for generating synthetic SQL queries based on schema information and sampling values from the underlying data. Hilprecht et al. [4] built on this approach by enabling the generation of more complex queries that might include joins, random index creation and predicates with null comparison, regex-based string comparison and disjunctions. Additionally, in the Master's thesis "Caching Strategies based on realistic Workloads," Martin Stemmer [14] investigates the practical, cost-based trade-off between full result caching and incremental updates in modern data warehouses.

---

## 3. Design

---

This chapter presents the design of the proposed ML-based view advisor. It first states the concrete objectives and system requirements derived from the problem statement, and then describes the proposed approach: system architecture, feature engineering, cost-model development and training, and the advisor integration and evaluation plan.

---

### 3.1. Objectives

---

The primary objective of this thesis is to build a machine learning advisor that recommends the best view strategy for an arbitrary SQL workload: (i) materialized view (MV) with full recomputation, (ii) incrementally maintained materialized view (IMV), or (iii) no view. To achieve this objective the following requirements are identified:

1. **Performance metrics.** Define clear, measurable metrics that capture both *view maintenance cost* and *query execution time* so that different strategies can be compared on the same basis.
2. **Dataset curation.** Produce a labeled dataset of query execution times for reads and writes across the three view configurations (MV, IMV, no view). Each dataset entry must include the query, its QEP(s), and measured execution times under controlled experimental conditions.
3. **Feature extraction.** Provide a pipeline-based feature extractor that maps QEPs into numerical feature vectors for both read and write queries.
4. **Write cost model.** Design and train a decision-tree-based model (LightGBM) to predict per-tuple maintenance time for write pipelines; the read-cost model is assumed available.
5. **Advisor logic.** Combine read- and write-cost predictions to estimate total workload time and produce a lowest-cost recommendation (MV / IMV / no view).
6. **Evaluation.** Validate the advisor on unseen workloads and measure (a) prediction accuracy of the cost models, and (b) the advisor’s success rate at selecting the true optimal strategy.

---

### 3.2. Proposed Approach

---

To address the research questions posed in this thesis, we adopt a learning-based, workload-driven approach. The design is organized into four main phases: (1) system architecture, (2) featurization of SQL queries, (3) development and training of cost models, and (4) advisor integration and evaluation.

---

### 3.2.1. System Architecture

The proposed system comprises three primary components (see Figure ??):

- **Feature extractor.** Parses a query's Query Execution Plan (QEP) and decomposes it into a set of pipelines. For each pipeline the extractor builds a numerical feature vector that summarizes operator-level properties, pipeline flow statistics, and relevant schema/table statistics.
- **Cost models.** Two specialized models are used:
  - *Read cost model:* predicts the execution time of SELECT queries under each view configuration. A pre-existing read-cost model will be reused for this component.
  - *Write cost model:* predicts the execution time of write queries (INSERT, UPDATE, DELETE) with an emphasis on per-tuple maintenance cost for view maintenance pipelines.
- **View advisor.** Given a workload (a sequence or distribution of SQL statements), the advisor queries the cost models for each statement under the three view scenarios, sums predicted pipeline costs to obtain per-statement and per-workload totals, and recommends the strategy with minimal predicted total workload time.

### 3.2.2. Feature Engineering

Feature engineering is split by query type (read vs. write) and follows a pipeline-centric decomposition.

**Read queries** For reads, we adopt the feature set and extraction procedures inspired by the T3 system [12]. Features are derived from the QEP and include:

- Operator-level attributes: operator type, estimated and actual output cardinalities, estimated cost, operator-specific metrics (e.g., join method).
- Table/column statistics: table size, column data types, histograms or distribution summaries where available.
- Pipeline-level aggregates: number of operators, depth of pipeline, estimated pipeline cost.

These features are designed to capture computational complexity and to enable the read-cost model to predict whether a view will accelerate the query.

**Write queries** For writes we use a similar pipeline-based representation but focus on tuple-centric and maintenance-relevant features:

- Tuple-flow features: fraction/percentage of tuples from the pipeline start that reach particular operators (selectivity estimates).
- Operator-level features: operator cardinalities, output row width, estimated intermediate sizes.
- View-related features: number of joins and aggregations present in the view definition, and expected delta sizes that the maintenance pipeline must process.
- Physical costs: index lookup counts, number of affected partitions/pages where measurable.

---

Additionally, execution plans will be used to extract and treat distinct pipelines separately so the model can learn pipeline-specific per-tuple costs.

### 3.2.3. Cost Model Development and Training

The write cost model will be implemented as a LightGBM gradient-boosted decision-tree ensemble. The model training follows these principles:

- **Training objective:** predict per-tuple processing time for a pipeline. The per-tuple formulation enables scaling predictions to arbitrary input cardinalities by multiplication with predicted/actual cardinalities at inference time.
- **Target transformation:** to stabilize training across a wide dynamic range of per-tuple times, the target per-tuple time  $t$  will be transformed via:

$$t' = -\log(t).$$

- **Training data:** the dataset is curated by running workloads (mixes of reads and writes) under the three view configurations and recording per-pipeline execution times, cardinalities, and QEP-derived features. The write dataset will include reconstructed/extracted workloads (e.g., the reconstructed Amazon Redshift workload referenced in ??).
- **Evaluation metrics:** model accuracy will be measured using metrics such as Mean Absolute Percentage Error (MAPE) and Q-Error on a held-out test set, and compared against baseline heuristics (e.g., simple cardinality-based rules).
- **Deployment optimization:** the trained LightGBM ensemble will be compiled to native code (or efficiently embedded) to enable fast inference during advisor operation.

### 3.2.4. Advisor Integration and Evaluation

The advisor integrates the read- and write-cost models to recommend a view strategy for a workload. The per-workload decision flow is:

1. For each statement in the workload, obtain its QEP in each scenario: MV, IMV, and no view.
2. Decompose each QEP into pipelines and extract feature vectors via the feature extractor.
3. Predict per-pipeline per-tuple time using the appropriate cost model (read or write) and scale by predicted/actual pipeline cardinalities to obtain pipeline times.
4. Sum pipeline times to obtain per-statement predicted times for each scenario.
5. Aggregate per-statement times across the workload to compute total predicted workload time for MV, IMV, and no view.
6. Recommend the strategy with the minimum predicted total workload time.



---

**Evaluation plan** End-to-end experiments will validate the advisor by executing a suite of test workloads on the target database. For each workload:

- Measure the *actual* total execution time under each strategy (MV, IMV, no view).
- Compare the advisor’s recommendation against the empirically best strategy. The primary success metric is the advisor’s accuracy: the percentage of workloads for which it selected the best-performing strategy.
- Report secondary metrics including the advisor’s predicted vs. actual total times, model error statistics (MAPE, Q-Error), and sensitivity analyses across workload mixes (read-heavy vs. write-heavy).

The next chapter describes implementation details and the experimental setup used to collect the dataset and run the evaluation.

---

## 4. Implementation

---

---

### 4.1. Implementation Overview

---

The implementation will realize the design above. Key implementation decisions described in the objectives include:

- Using LightGBM for the write-cost decision-tree ensemble and compiling the resulting model for fast inference.
- Reusing the existing read-cost model and its feature extraction pipeline.
- Instrumenting the database/testing infrastructure to record per-statement execution times and execution plans across the three view configurations (no view, MV, IMV) to build the training dataset.
- Implementing an advisor component that consumes predicted read/write costs and applies the decision logic to produce a recommendation.

Further implementation details (choice of database system, workload generator parameters, exact instrumentation methods, and integration scripts) will be documented in Chapter 4 as they are finalized and implemented.

---

## 5. Evaluation

---

To evaluate the performance and usefulness of...

---

### 5.1. Experimental setup

---

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

---

## 6. Conclusion and Future Work

---

This chapter first summarizes and concludes the contributions of this thesis by presenting the strengths behind the design concepts and concluding upon the evaluation. Following, an outlook of research challenges for future work is given.

---

### 6.1. Conclusion

---

Mirror the contributions given in intro.

Conclude on the evaluation Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

---

### 6.2. Future Work

---

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

---

## Bibliography

---

- [1] Rafi Ahmed et al. “Automated generation of materialized views in oracle”. In: *Proceedings of the VLDB Endowment* 13.12 (2020), pp. 3046–3058.
- [2] Mert Akdere et al. “Learning-based query performance modeling and prediction”. In: *2012 IEEE 28th International Conference on Data Engineering*. IEEE. 2012, pp. 390–401.
- [3] Chetan Gupta, Abhay Mehta, and Umeshwar Dayal. “PQR: Predicting query execution times for autonomous workload management”. In: *2008 International Conference on Autonomic Computing*. IEEE. 2008, pp. 13–22.
- [4] Benjamin Hilprecht and Carsten Binnig. “Zero-shot cost models for out-of-the-box learned cost prediction”. In: *arXiv preprint arXiv:2201.00561* (2022).
- [5] Benjamin Hilprecht, Carsten Binnig, and Uwe Röhm. “Learning a partitioning advisor for cloud databases”. In: *Proceedings of the 2020 ACM SIGMOD international conference on management of data*. 2020, pp. 143–157.
- [6] Benjamin Hilprecht et al. “DBMS Fitting: Why should we learn what we already know?” In: *CIDR*. 2020.
- [7] Andreas Kipf et al. “Learned cardinalities: Estimating correlated joins with deep learning”. In: *arXiv preprint arXiv:1809.00677* (2018).
- [8] Jan Kossmann, Alexander Kastius, and Rainer Schlosser. “SWIRL: Selection of Workload-aware Indexes using Reinforcement Learning.” In: *EDBT*. Vol. 2. 2022, pp. 155–2.
- [9] Claude Lehmann, Pavel Sulimov, and Kurt Stockinger. “Is your learned query optimizer behaving as you expect? a machine learning perspective”. In: *arXiv preprint arXiv:2309.01551* (2023).
- [10] Sherry Listgarten and Marie-Anne Neimat. “Cost model development for a main memory database system”. In: *Real-Time Database Systems: Issues and Applications*. Springer, 1997, pp. 139–162.
- [11] Stefan Manegold, Peter Boncz, and Martin L Kersten. “Generic database cost models for hierarchical memory systems”. In: *VLDB’02: Proceedings of the 28th International Conference on Very Large Databases*. Elsevier. 2002, pp. 191–202.
- [12] Maximilian Rieger and Thomas Neumann. “T3: Accurate and Fast Performance Prediction for Relational Database Systems With Compiled Decision Trees”. In: *Proceedings of the ACM on Management of Data* 3.3 (2025), pp. 1–27.
- [13] P Griffiths Selinger et al. “Access path selection in a relational database management system”. In: *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*. 1979, pp. 23–34.
- [14] Martin Stemmer. “Caching Strategies based on realistic Workloads”. Date of submission: May 22, 2025. MA thesis. Technische Universität Darmstadt, May 2025.

- 
- [15] Ji Sun and Guoliang Li. “An end-to-end learning-based cost estimator”. In: *arXiv preprint arXiv:1906.02560* (2019).
  - [16] Wentao Wu et al. “Predicting query execution time: Are optimizer cost models really unusable?”. In: *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. IEEE. 2013, pp. 1081–1092.
  - [17] Wentao Wu et al. “Towards predicting query execution time for concurrent and dynamic database workloads”. In: *Proceedings of the VLDB Endowment* 6.10 (2013), pp. 925–936.
  - [18] Ziniu Wu et al. “Stage: Query execution time prediction in amazon redshift”. In: *Companion of the 2024 International Conference on Management of Data*. 2024, pp. 280–294.
  - [19] Jiani Yang et al. “Rethinking Learned Cost Models: Why Start from Scratch?”. In: *Proceedings of the ACM on Management of Data* 1.4 (2023), pp. 1–27.
  - [20] Haitao Yuan et al. “Automatic view generation with deep learning and reinforcement learning”. In: *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE. 2020, pp. 1501–1512.
  - [21] Yue Zhao et al. “Queryformer: A tree transformer model for query plan representation”. In: *Proceedings of the VLDB Endowment* 15.8 (2022), pp. 1658–1670.
  - [22] Xuanhe Zhou et al. “Query performance prediction for concurrent queries using graph embedding”. In: *Proceedings of the VLDB Endowment* 13.9 (2020), pp. 1416–1428.

---

## A. Experiment Parameters

---

The experiments conducted in the Evaluation Chapter were executed with the following DPI parameters. Note, an  $x$  indicates the parameter has been varied for the experiment.

|        | Message size | Segment size | Ring size (segments) | Internal output buffer size |
|--------|--------------|--------------|----------------------|-----------------------------|
| Exp. A | $x$          | 64 MiB       | 10                   | 8 MiB                       |
| Exp. B | $x$          | $x$          | 100                  | 4 MiB                       |
| Exp. C | 4 KiB        | 64 MiB       | 10                   | $x$                         |
| Exp. D | 4 KiB        | 32 MiB       | $x$                  | 4 MiB                       |
| Exp. E | 4 KiB        | $x$          | $x$                  | 4 MiB                       |
| Exp. F | $x$          | 32 MiB       | 50                   | 8 MiB                       |
| Exp. G | $x$          | 64 MiB       | 50                   | 4 MiB                       |
| Exp. H | 8 KiB        | 4 MiB        | $x$                  | 2 MiB                       |